

1. Copy and run the following code to construct some toy data

Solution.

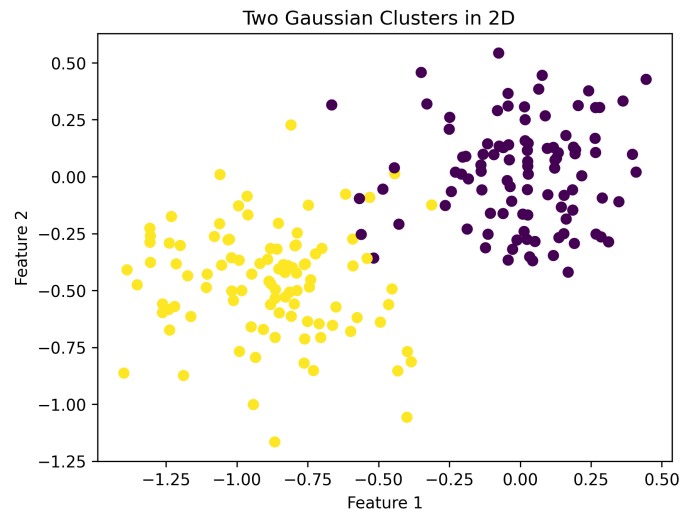


Figure 1: Two Gaussian clusters in 2D with labels -1 and $+1$.

2. The code below is an example of a SVM classifier. By using it, write a code that trains a linear SVM for classification. Represent in the same graph the classification boundary, the margins and the support vectors.

Solution.

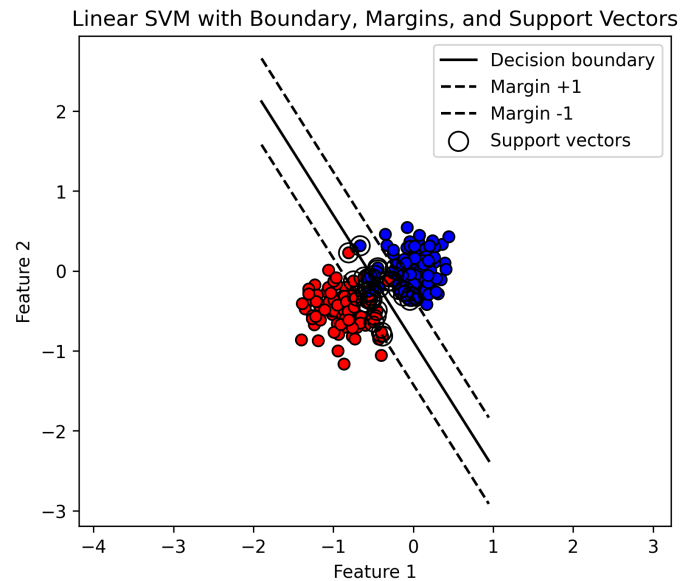


Figure 2: Linear SVM classification boundary, margins, and support vectors.

3. The following code generates a set of clusters (N_{clusters}) in a space of N_{dim} dimensions. Each cluster contains data with labels $+1$ or -1 . The number of dimensions must be at least equal to the number of clusters minus one for the data to be linearly separable.

3.1 Find the value of C that produces the best validation error. Repeat the experiment 100 times with different training and validation data and average the values of the optimal C (C_{opt}) and the corresponding averaged error (E_{opt}). The error must be defined as the fraction of misclassified samples (not MSE).

Perform the experiment for the following configurations:

- 8 clusters, 7 dimensions, $N_{\text{train}} = 2 \times N_{\text{clusters}}$
- 8 clusters, 70 dimensions, $N_{\text{train}} = 2 \times N_{\text{clusters}}$
- 8 clusters, 7 dimensions, $N_{\text{train}} = 100 \times N_{\text{clusters}}$
- 8 clusters, 70 dimensions, $N_{\text{train}} = 100 \times N_{\text{clusters}}$

Use $N_{\text{val}} = 100 \times N_{\text{clusters}}$. Report the averaged optimal C and error values.

Solution.

For each configuration, we generated random orthogonal clusters, trained a linear SVM over a logarithmic grid of C values, and selected the one minimizing the validation misclassification rate. The procedure was repeated 100 times, and the averages of C_{opt} and E_{opt} were computed.

N_{dim}	N_{train}	$\langle C_{\text{opt}} \rangle$	$\langle E_{\text{opt}} \rangle$
7	16	0.7820	0.0091
70	16	0.0316	0.0000
7	800	0.7467	0.0043
70	800	0.0316	0.0000

Table 1: Average optimal C and validation error over 100 independent runs for different dimensional and training data configurations.

3.2 Provide a compelling answer to the following questions:

- (a) If the number of dimensions increases, does C_{opt} increase or decrease? Why?

Solution.

As the number of dimensions increases, the data become more linearly separable, so the optimal regularization parameter C_{opt} tends to **decrease**. A smaller C is sufficient because higher dimensions make it easier to separate classes without misclassification.

- (b) For the same number of training data, do you find any significant differences in the validation errors for 7 and 70 dimensions? Why?

Solution.

For the same number of training samples, the validation errors for 7 and 70 dimensions are both very low. Since the data are synthetically well-separated, increasing dimensionality does not worsen performance—in fact, the 70-dimensional case often achieves perfect separation.

Appendix

Listing 1: Homework Code

```

1 import os
2 import matplotlib.pyplot as plt
3 import numpy as np
4 from sklearn.datasets import make_blobs
5 from sklearn import svm
6 from sklearn.svm import LinearSVC
7 from numpy.linalg import eig
8
9 # Problem 1
10
11 N_dimensions=2
12 center_1=np.zeros(N_dimensions)
13 center_2=np.random.randn(N_dimensions)
14 center_2=center_2/np.linalg.norm(center_2)
15
16 X, y = make_blobs(n_samples=200, centers=[center_1, center_2], cluster_std=0.25)
17 y=2*(y-1/2)
18
19 plt.scatter(X[:, 0], X[:, 1], c=y)
20 plt.title('Two Gaussian Clusters in 2D ')
21 plt.xlabel('Feature 1')
22 plt.ylabel('Feature 2')
23
24 os.makedirs("plots", exist_ok=True)
25 plt.savefig("plots/p1.png", dpi=300, bbox_inches="tight")
26 plt.show()
27
28 # Problem 2
29
30 clf = svm.SVC(kernel='linear')
31 clf.fit(X, y)
32 y_pred = clf.predict(X)
33
34 w = clf.coef_[0]
35 b = clf.intercept_[0]
36
37 plt.figure(figsize=(6,5))
38 plt.scatter(X[:, 0], X[:, 1], c=y, cmap='bwr', edgecolors='k', s=40)
39
40 xx = np.linspace(X[:, 0].min()-0.5, X[:, 0].max()+0.5, 200)
41 yy = (-w[0] * xx - b) / w[1]
42 yy_plus = (-w[0] * xx - b + 1) / w[1]
43 yy_minus = (-w[0] * xx - b - 1) / w[1]
44
45 plt.plot(xx, yy, 'k-', label='Decision boundary')
46 plt.plot(xx, yy_plus, 'k--', label='Margin +1')
47 plt.plot(xx, yy_minus, 'k--', label='Margin -1')
48
49 plt.scatter(

```

```

50 clf.support_vectors_[:, 0],
51 clf.support_vectors_[:, 1],
52 s=120, facecolors='none', edgecolors='k', label='Support vectors'
53 )
54
55 plt.title('Linear SVM with Boundary, Margins, and Support Vectors')
56 plt.xlabel('Feature 1')
57 plt.ylabel('Feature 2')
58 plt.legend(loc='best')
59 plt.axis('equal')
60
61 plt.savefig("plots/p2.png", dpi=300, bbox_inches="tight")
62 plt.show()
63
64 # Problem 3.1
65
66 def data(T, N_data, N_dimensions):
67     sigma = 0.15
68     centers = np.concatenate(
69         (np.ones((1, T.shape[0])), np.eye(T.shape[0])),
70         axis=0
71     ) @ T
72     X, y = make_blobs(
73         n_samples=N_data,
74         centers=centers,
75         cluster_std=np.sqrt(sigma) / np.sqrt(N_dimensions)
76     )
77     y[np.where(np.mod(y, 2) == 1)] = -1
78     y[np.where(np.mod(y, 2) == 0)] = 1
79     return X, y
80
81 def run_once(N_clusters, N_dim, N_train, N_val, C_grid):
82     A = np.random.randn(N_dim, N_dim)
83     _, V = eig(A @ A.T)
84     T = V[0:N_clusters-1, :]
85
86     X_train, y_train = data(T, N_train, N_dim)
87     X_val, y_val = data(T, N_val, N_dim)
88
89     best_C, best_err = None, 1.0
90
91     for C in C_grid:
92         clf = LinearSVC(C=C, max_iter=2000, dual=False)
93         clf.fit(X_train, y_train)
94         y_pred = clf.predict(X_val)
95         err = np.mean(y_pred != y_val)
96         if err < best_err:
97             best_err, best_C = err, C
98
99     return best_C, best_err
100
101 N_clusters = 8
102 N_val = 100 * N_clusters
103 C_grid = np.logspace(-1.5, 1, 15)

```

```

104 N_repeats = 100
105
106 setups = [
107     (8, 7, 2*8),
108     (8, 70, 2*8),
109     (8, 7, 100*8),
110     (8, 70, 100*8),
111 ]
112
113 results = []
114
115 for (Nc, Nd, Ntr) in setups:
116     print(f"\nRunning setup: N_dim={Nd}, N_train={Ntr} ...")
117     C_list, E_list = [], []
118
119     for i in range(N_repeats):
120         C_opt, E_opt = run_once(Nc, Nd, Ntr, N_val, C_grid)
121         C_list.append(C_opt)
122         E_list.append(E_opt)
123
124     pct = ((i + 1) / N_repeats) * 100
125     print(f"Progress: {pct:6.2f}% ({i+1}/{N_repeats})", end="\r")
126
127     print()
128     results.append({
129         "N_dim": Nd,
130         "N_train": Ntr,
131         "C_opt_avg": np.mean(C_list),
132         "E_opt_avg": np.mean(E_list)
133     })
134
135     print("\nAverage results over", N_repeats, "runs:")
136     print(" N_dim | N_train | <C_opt> | <E_opt>")
137     print("-----")
138     for r in results:
139         print(f"{r['N_dim']:6d} | {r['N_train']:8d} | {r['C_opt_avg']:9.4f} | {r['E_opt_avg']:.4f}")
140

```